

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG VỚI LỚP TRONG C#

Thời lượng: 3 tiết

YÊU CẦU NỘP BÀI

Nội dung mỗi tiết được thực hiện thông qua một dự án. Sau khi hoàn thành dự án, nén file *.cs và nộp vào Bài tập tương ứng trên Canvas.

I. MỤC TIÊU

Sau khi hoàn thành bài thực hành, sinh viên có khả năng:

- **Hiểu và định nghĩa** được lớp (class) trong C#.
- **Tạo và sử dụng** các thành phần của lớp: hàm khởi tạo (constructor), hàm hủy (destructor), phương thức (method), thuộc tính (property).
- **Vận dụng** các tính chất của lập trình hướng đối tượng:
 - Tính kế thừa (Inheritance).
 - Tính đa hình (Polymorphism) thông qua phương thức ảo (virtual method) và ghi đè phương thức (override).
 - Chồng phương thức (Method Overloading).
- **Xây dựng thuộc tính đúng chuẩn**, có kiểm soát dữ liệu đầu vào.
- **Áp dụng** kiến thức về lớp để xây dựng một ứng dụng quản lý đơn giản.

II. YÊU CẦU

- Máy tính đã cài đặt Visual Studio (phiên bản 2019 trở lên).
- Nắm vững kiến thức cơ bản về lập trình C# (biến, kiểu dữ liệu, toán tử, cấu trúc điều khiển, mảng).

III. NỘI DUNG THỰC HÀNH

TIẾT 1: KHÁI NIỆM CƠ BẢN VỀ LỚP VÀ ĐỐI TƯỢNG (50 PHÚT)

A. Lý thuyết ôn tập (10 phút)

- **Lớp (Class):** Là một bản thiết kế (khuôn mẫu) để tạo ra các đối tượng. Lớp định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi) mà các đối tượng của lớp đó sẽ có.
- **Đối tượng (Object):** Là một thể hiện cụ thể của một lớp, được tạo ra trong bộ nhớ.
- **Hàm khởi tạo (Constructor):** Là một phương thức đặc biệt được gọi tự động khi một đối tượng được tạo ra. Dùng để khởi tạo giá trị ban đầu cho các thuộc tính của đối tượng.
- **Hàm hủy (Destructor):** Là phương thức đặc biệt, được gọi tự động bởi Garbage Collector khi đối tượng bị thu hồi. Dùng để giải phóng tài nguyên. Trong C# hiện đại, chúng ta ít khi cần tự định nghĩa hàm hủy.
- **Phương thức (Method):** Định nghĩa các hành vi, hành động mà đối tượng có thể thực hiện.

B. Bài tập 1: Xây dựng lớp SinhVien cơ bản (40 phút)

Yêu cầu: Xây dựng một lớp SinhVien để lưu trữ thông tin cơ bản của một sinh viên và hiển thị thông tin đó ra màn hình.

Hướng dẫn:

1. **Tạo Project:** Mở Visual Studio, tạo một project mới thuộc loại **Console App** và đặt tên là QuanLySinhVien.
2. **Tạo lớp SinhVien:**
 - Trong cửa sổ Solution Explorer, chuột phải vào tên project, chọn **Add > Class....**
 - Đặt tên file là SinhVien.cs và nhấn **Add**.
3. Định nghĩa các trường (fields):

Trong file SinhVien.cs, thêm các trường để lưu trữ thông tin. Hiện tại, chúng ta dùng public để truy cập trực tiếp.

```
class SinhVien
{
    // Các trường dữ liệu (fields)
    public string maSV;
    public string hoTen;
    public int namSinh;
    public double diemTB;
}
```

4. Tạo hàm khởi tạo:

Thêm hai hàm khởi tạo: một hàm không tham số (default constructor) và một hàm có tham số để khởi tạo đối tượng với đầy đủ thông tin.

```
class SinhVien
{
    // ... (phần fields ở trên)

    // Hàm khởi tạo không tham số
    public SinhVien()
    {
        maSV = "22IT.000";
        hoTen = "Chưa có tên";
        namSinh = 2004;
        diemTB = 0;
    }

    // Hàm khởi tạo có tham số
    public SinhVien(string ma, string ten, int ns, double dtb)
    {
        maSV = ma;
        hoTen = ten;
        namSinh = ns;
        diemTB = dtb;
    }
}
```

```
}
```

5. Tạo phương thức:

Thêm phương thức `HienThiThôngTin()` để in thông tin sinh viên ra màn hình.

```
class SinhVien
{
    // ... (phần fields và constructors ở trên)

    // Phương thức hiển thị thông tin
    public void HienThiThôngTin()
    {
        Console.WriteLine("--- Thông tin sinh viên ---");
        Console.WriteLine($"Ma SV: {maSV}");
        Console.WriteLine($"Ho ten: {hoTen}");
        Console.WriteLine($"Nam sinh: {namSinh}");
        Console.WriteLine($"Diem TB: {diemTB}");
    }
}
```

6. Sử dụng lớp `SinhVien`:

Mở file `Program.cs`, trong hàm `Main`, tạo các đối tượng `SinhVien` và gọi phương thức của chúng.

```
class Program
{
    static void Main(string[] args)
    {
        // Tạo đối tượng sv1 bằng hàm khởi tạo không tham số
        SinhVien sv1 = new SinhVien();
        Console.WriteLine("Thông tin SV1:");
        sv1.HienThiThôngTin();

        Console.WriteLine("\n");

        // Tạo đối tượng sv2 bằng hàm khởi tạo có tham số
        SinhVien sv2 = new SinhVien("22IT.123", "Nguyen Van A", 2004, 8.5);
        Console.WriteLine("Thông tin SV2:");
        sv2.HienThiThôngTin();

        Console.ReadKey();
    }
}
```

7. **Chạy chương trình:** Nhấn F5 để biên dịch và chạy. Quan sát kết quả.

TIẾT 2: TÍNH CHẤT NÂNG CAO - THUỘC TÍNH, KẾ THỪA, ĐA HÌNH (50 PHÚT)

A. Lý thuyết ôn tập (10 phút)

- **Thuộc tính (Property):** Cung cấp một cơ chế linh hoạt để đọc, ghi hoặc tính toán giá trị của một trường private. Giúp kiểm soát truy cập và xác thực dữ liệu.
- **Kế thừa (Inheritance):** Cho phép một lớp (lớp con - derived class) thừa hưởng các đặc điểm (trường, phương thức) từ một lớp khác (lớp cha - base class).
- **Phương thức ảo (Virtual Method):** Là phương thức ở lớp cha có từ khóa virtual, cho phép lớp con có thể định nghĩa lại (ghi đè - override) phương thức đó.
- **Đa hình (Polymorphism):** "Một tên, nhiều hình thái". Cho phép một đối tượng có thể được tham chiếu bởi kiểu của lớp cha nhưng khi gọi phương thức bị ghi đè, phiên bản của lớp con sẽ được thực thi.

B. Bài tập 2: Cải tiến lớp SinhVien và áp dụng kế thừa (40 phút)

Yêu cầu:

- Chuyển các trường public của lớp SinhVien thành private và sử dụng thuộc tính (property) để truy cập.
- Thêm ràng buộc cho thuộc tính NamSinh (ví dụ: phải lớn hơn 1950).
- Tạo lớp cha Nguoi và cho lớp SinhVien kế thừa.
- Áp dụng phương thức ảo và ghi đè.

Hướng dẫn:

1. Tạo lớp cha Nguoi:

- Tạo một file class mới tên là Nguoi.cs.
- Định nghĩa lớp Nguoi với các thông tin chung.

// Trong file Nguoi.cs

```
class Nguoi
```

```
{
```

```
    public string HoTen { get; set; }
```

```
    public int NamSinh { get; set; }
```

```
    public Nguoi() {}
```

```
    public Nguoi(string ten, int ns)
```

```
    {
```

```
        HoTen = ten;
```

```
        NamSinh = ns;
```

```
    }
```

```
// Đánh dấu phương thức này là ảo để lớp con có thể ghi đè
```

```
public virtual void HienThiThongTin()
```

```
{
```

```
    Console.WriteLine($"Ho ten: {HoTen}");
```

```
    Console.WriteLine($"Nam sinh: {NamSinh}");
```

```
}
```

```
}
```

2. Cập nhật lớp SinhVien:

- Mở file SinhVien.cs.
- Cho SinhVien kế thừa từ Nguoi.
- Chuyển các trường thành private và tạo public property để truy cập.

// Trong file SinhVien.cs

```
class SinhVien : Nguoi // Kế thừa từ lớp Nguoi
```

```
{
```

```
    // Các trường dữ liệu private
```

```
    private string _maSV;
```

```
    private double _diemTB;
```

```
    // Thuộc tính (Property) cho maSV
```

```
    public string MaSV
```

```
    {
```

```
        get { return _maSV; }
```

```
        set { _maSV = value; }
```

```
    }
```

```
    // Thuộc tính cho diemTB
```

```
    public double DiemTB
```

```
    {
```

```
        get { return _diemTB; }
```

```
        set
```

```
        {
```

```
            // Ràng buộc dữ liệu: điểm phải từ 0 đến 10
```

```
            if (value >= 0 && value <= 10)
```

```
            {
```

```
                _diemTB = value;
```

```
            }
```

```
            else
```

```
            {
```

```
                // In ra cảnh báo nếu điểm không hợp lệ
```

```
                Console.WriteLine("Diem khong hop le!");
```

```
            }
```

```
        }
```

```
    }
```

```
    // Ghi đè (override) phương thức của lớp cha
```

```
    public override void HienThiThongTin()
```

```
    {
```

```
        Console.WriteLine("--- Thong tin sinh vien ---");
```

```
        base.HienThiThongTin(); // Gọi phương thức của lớp cha để hiển thị thông tin chung
```

```
        Console.WriteLine($"Ma SV: {MaSV}");
```

```
        Console.WriteLine($"Diem TB: {DiemTB}");
```

```
    }
```

```

// Cập nhật hàm khởi tạo
public SinhVien(string ma, string ten, int ns, double dtb) : base(ten, ns) // Gọi hàm khởi tạo của
lớp cha
{
    MaSV = ma;
    DiemTB = dtb;
}
}

```

3. Sử dụng trong Program.cs:

- Cập nhật hàm Main để sử dụng các thuộc tính và thể hiện tính đa hình.

```

class Program
{
    static void Main(string[] args)
    {
        // Tạo đối tượng SinhVien
        SinhVien sv = new SinhVien("22IT.456", "Tran Thi B", 2003, 9.2);

        // Thể hiện tính đa hình: một biến kiểu Nguoi có thể tham chiếu đến một đối tượng
        SinhVien
        Nguoi nguoi = sv;

        Console.WriteLine("Thông tin hiển thị từ biến kiểu Nguoi:");
        nguoi.HienThiThongTin(); // Phương thức HienThiThongTin() của SinhVien sẽ được gọi

        Console.WriteLine("\n");

        // Thử gán giá trị không hợp lệ cho thuộc tính DiemTB
        Console.WriteLine("Gán diemTB = 11");
        sv.DiemTB = 11;
        sv.HienThiThongTin(); // DiemTB vẫn giữ giá trị cũ (9.2)

        Console.ReadKey();
    }
}

```

TIẾT 3: ỨNG DỤNG QUẢN LÝ DANH SÁCH SINH VIÊN (50 PHÚT)

A. Lý thuyết ôn tập (5 phút)

- **List<T>**: Là một cấu trúc dữ liệu trong .NET cho phép lưu trữ một danh sách các đối tượng có cùng kiểu T. List rất linh hoạt, cho phép thêm, xóa, sửa, tìm kiếm các phần tử một cách dễ dàng.

B. Bài tập 3: Xây dựng chương trình quản lý lớp học đơn giản (45 phút)

Yêu cầu: Xây dựng một ứng dụng console cho phép người dùng thực hiện các chức năng:

1. Nhập danh sách sinh viên.
2. Xuất danh sách sinh viên.

}

```

        Console.WriteLine("\nNhan phim bat ky de tiep tục...");
        Console.ReadKey();
    }
}

static void ThemSinhVien(List<SinhVien> ds)
{
    Console.WriteLine("\n--- Them sinh vien moi ---");
    Console.Write("Nhap ma SV: ");
    string ma = Console.ReadLine();

    Console.Write("Nhap ho ten: ");
    string ten = Console.ReadLine();

    Console.Write("Nhap nam sinh: ");
    int ns = int.Parse(Console.ReadLine());

    Console.Write("Nhap diem TB: ");
    double dtb = double.Parse(Console.ReadLine());

    SinhVien sv = new SinhVien(ma, ten, ns, dtb);
    ds.Add(sv); // Thêm sinh viên vào danh sách

    Console.WriteLine("=> Da them thanh công!");
}

static void HienThiDanhSach(List<SinhVien> ds)
{
    Console.WriteLine("\n--- Danh sach sinh vien ---");
    if (ds.Count == 0)
    {
        Console.WriteLine("Danh sach rong!");
        return;
    }

    foreach (SinhVien sv in ds)
    {
        sv.HienThiThongTin();
        Console.WriteLine("-----");
    }
}
}

```


2. Chạy và kiểm thử:

- Nhấn F5 để chạy chương trình.
- Chọn chức năng 1 để thêm một vài sinh viên.
- Chọn chức năng 2 để xem lại danh sách vừa nhập.
- Chọn chức năng 3 để thoát.

IV. BÀI TẬP MỞ RỘNG (NẾU CÒN THỜI GIAN)

1. Thêm chức năng **tìm kiếm sinh viên** theo mã SV.
2. Thêm chức năng **xóa sinh viên** khỏi danh sách.
3. Thêm chức năng **sắp xếp danh sách sinh viên** theo điểm trung bình tăng dần/giảm dần.
4. Tạo thêm lớp **GiangVien** kế thừa từ lớp **Nguoi** và xây dựng chương trình quản lý chung cho cả **SinhVien** và **GiangVien** (sử dụng `List<Nguoi>`).

V. TỔNG KẾT

Buổi thực hành đã giúp ôn tập và vận dụng các kiến thức cốt lõi của lập trình hướng đối tượng trong C#. Từ việc tạo lớp, sử dụng các thành phần cơ bản, cho đến việc áp dụng các tính chất quan trọng như kế thừa, đa hình vào một bài toán thực tế. Đây là nền tảng vững chắc để tiếp cận các công nghệ phức tạp hơn trong .NET.